

Digital Design: Introduction to digital logic circuits.

COMBINATIONAL CIRCUITS:

INTRODUCTION:

The digital system consists of two types of circuits, namely

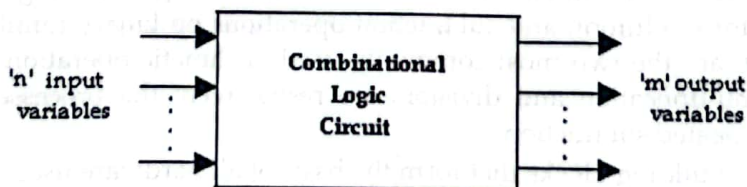
- Combinational circuits
- Sequential circuits

Combinational circuit consists of logic gates whose output at any time is determined from the present combination of inputs. The logic gate is the most basic building block of combinational logic. The logical function performed by a combinational circuit is fully defined by a set of Boolean expressions.

Sequential logic circuit comprises both logic gates and the state of storage elements such as flip-flops. As a consequence, the output of a sequential circuit depends not only on present value of inputs but also on the past state of inputs.

In the previous chapter, we have discussed binary numbers, codes, Boolean algebra and simplification of Boolean function and logic gates. In this chapter, formulation and analysis of various systematic designs of combinational circuits will be discussed.

A combinational circuit consists of input variables, logic gates, and output variables. The logic gates accept signals from inputs and output signals are generated according to the logic circuits employed in it. Binary information from the given data transforms to desired output data in this process. Both input and output are obviously the binary signals, i.e., both the input and output signals are of two possible states, logic 1 and logic 0.



Block diagram of a combinational logic circuit

For n number of input variables to a combinational circuit, 2^n possible combinations of binary input states are possible. For each possible combination, there is one and only one possible output combination. A combinational logic circuit can be described by m Boolean functions and each output can be expressed in terms of n input variables.



Digital Design: Introduction to digital logic circuits.

COMBINATIONAL CIRCUITS:**INTRODUCTION:**

The digital system consists of two types of circuits, namely

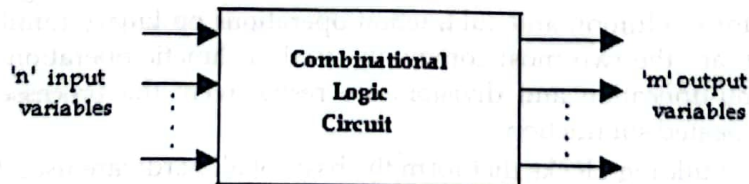
- (i) Combinational circuits
- (ii) Sequential circuits

Combinational circuit consists of logic gates whose output at any time is determined from the present combination of inputs. The logic gate is the most basic building block of combinational logic. The logical function performed by a combinational circuit is fully defined by a set of Boolean expressions.

Sequential logic circuit comprises both logic gates and the state of storage elements such as flip-flops. As a consequence, the output of a sequential circuit depends not only on present value of inputs but also on the past state of inputs.

In the previous chapter, we have discussed binary numbers, codes, Boolean algebra and simplification of Boolean function and logic gates. In this chapter, formulation and analysis of various systematic designs of combinational circuits will be discussed.

A combinational circuit consists of input variables, logic gates, and output variables. The logic gates accept signals from inputs and output signals are generated according to the logic circuits employed in it. Binary information from the given data transforms to desired output data in this process. Both input and output are obviously the binary signals, i.e., both the input and output signals are of two possible states, logic 1 and logic 0.



Block diagram of a combinational logic circuit

For n number of input variables to a combinational circuit, 2^n possible combinations of binary input states are possible. For each possible combination, there is one and only one possible output combination. A combinational logic circuit can be described by m Boolean functions and each output can be expressed in terms of n input variables.



dry = don't repeat your code

مراجعة

DESIGN PROCEDURE:

Any combinational circuit can be designed by the following steps of design procedure.

1. The problem is stated.
2. Identify the input and output variables.
3. The input and output variables are [assigned letter symbols]
4. Construction of a truth table to meet input-output requirements.
5. Writing Boolean expressions for various output variables in terms of input variables. $A + xz$
6. The simplified Boolean expression is obtained by any method of minimization – algebraic method, Karnaugh map method, or tabulation method.
7. A logic diagram is realized from the simplified boolean expression using logic gates. xy

The following guidelines should be followed while choosing the preferred form for hardware implementation:

1. [The implementation should have the minimum number of gates, with the gates used having the minimum number of inputs.]
2. There should be a minimum number of interconnections.
3. Limitation on the driving capability of the gates should not be ignored.

ARITHMETIC CIRCUITS - BASIC BUILDING BLOCKS:

In this section, we will discuss those combinational logic building blocks that can be used to perform addition and subtraction operations on binary numbers. Addition and subtraction are the two most commonly used arithmetic operations, as the other two, namely [multiplication and division], are respectively the processes of repeated addition and repeated subtraction.

The basic building blocks that form the basis of all hardware used to perform the arithmetic operations on binary numbers are half-adder, full adder, half-subtractor, full-subtractor.

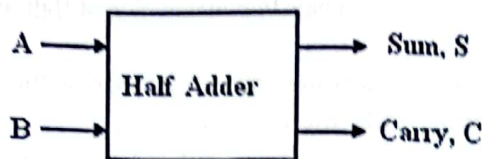
Half-Adder:

A half-adder is a combinational circuit that can be used to add (two binary bits). It has two inputs that represent the two bits to be added and two outputs, with one producing the SUM output and the other producing the CARRY.

2

x	y	z	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

procedure.



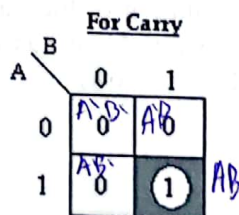
Block schematic of half-adder

The truth table of a half-adder, showing all possible input combinations and the corresponding outputs are shown below.

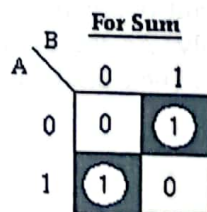
Inputs		Outputs	
A	B	Carry (C)	Sum (S)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Truth table of half-adder

K-map simplification for carry and sum:



Carry = A . B



Sum = $AB' + A'B$
 $= A \oplus B$

xor

0	0	→	0
0	1	→	1
1	0	→	1
1	1	→	0

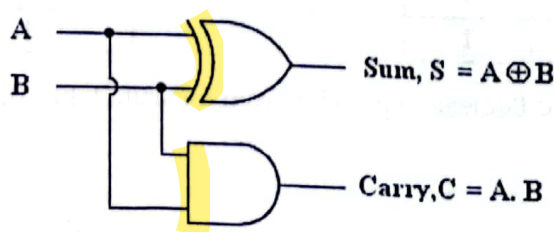
The Boolean expressions for the SUM and CARRY outputs are given by the equations,

Sum, S = $A'B + AB' = A \oplus B$

Carry, C = $A . B$

The first one representing the SUM output is that of an X-OR gate, the second one representing the CARRY output is that of an AND gate.

The logic diagram of the half adder is,



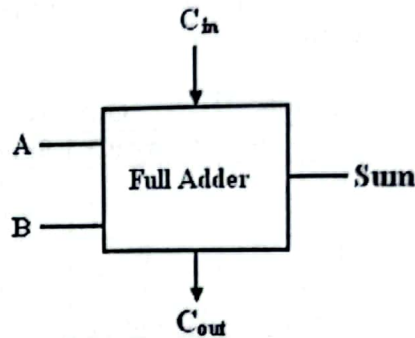
Half Adder

Logic Implementation of Half-adder

Full-Adder:

A full adder is a combinational circuit that forms the arithmetic sum of three input bits. It consists of 3 inputs and 2 outputs.

Two of the input variables, represent the significant bits to be added. The third input represents the carry from previous lower significant position. The block diagram of full adder is given by,



Block schematic of full-adder

The full adder circuit overcomes the limitation of the half-adder, which can be used to add two bits only. As there are three input variables, eight different input combinations are possible. The truth table is shown below,

Truth Table:

Inputs			Outputs	
A	B	C _{in}	Sum (S)	Carry (C _{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

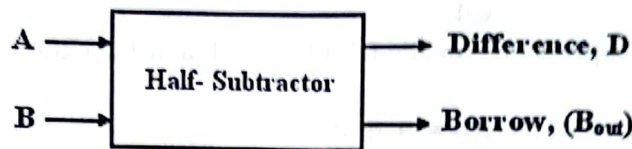
To derive the simplified Boolean expression from the truth table, the Karnaugh map method is adopted as,

4 Full Adder

Simply addition operation

Half-Subtractor:

A half-subtractor is a combinational circuit that can be used to subtract one binary digit from another to produce a DIFFERENCE output and a BORROW output. The BORROW output here specifies whether a '1' has been borrowed to perform the subtraction.



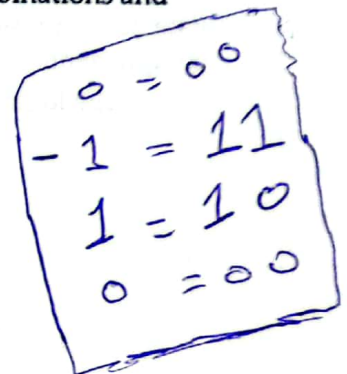
Block schematic of half-subtractor

The truth table of half-subtractor, showing all possible input combinations and the corresponding outputs are shown below.

Input		Output	
A	B	Difference (D)	Borrow (B _{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

ق.

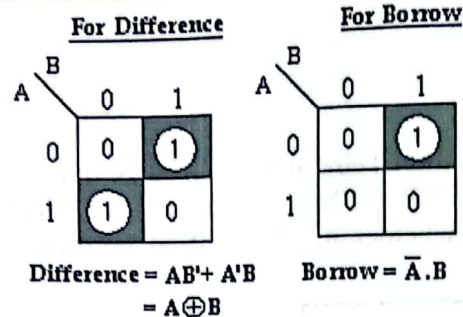
Need to be explained



فصله جده ولاد
يحيى ابي يحيى فحمنا

ull Subtractor:
A full subtractor
subtrahend, and also takes
by the previous adjacent
As a result
namely at

K-map simplification for half subtractor:



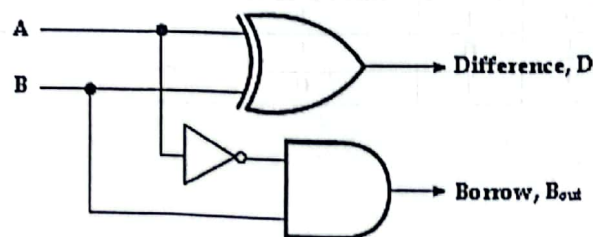
The Boolean expressions for the DIFFERENCE and BORROW outputs are given by the equations,

Difference, D = $A'B + AB' = A \oplus B$

Borrow, B_{out} = $A' \cdot B$

The first one representing the DIFFERENCE (D) output is that of an exclusive-OR gate, the expression for the BORROW output (B_{out}) is that of an AND gate with input A complemented before it is fed to the gate.

The logic diagram of the half subtractor is,



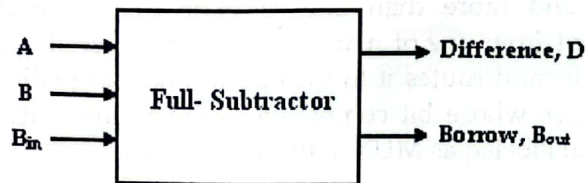
Logic Implementation of Half-Subtractor

Comparing a half-subtractor with a half-adder, we find that the expressions for the SUM and DIFFERENCE outputs are just the same. The expression for BORROW in the case of the half-subtractor is also similar to what we have for CARRY in the case of the half-adder. If the input A, ie., the minuend is complemented, an AND gate can be used to implement the BORROW output.

Full Subtractor:

A *full subtractor* performs subtraction operation on two bits, a minuend and a subtrahend, and also takes into consideration whether a '1' has already been borrowed by the previous adjacent lower minuend bit or not.

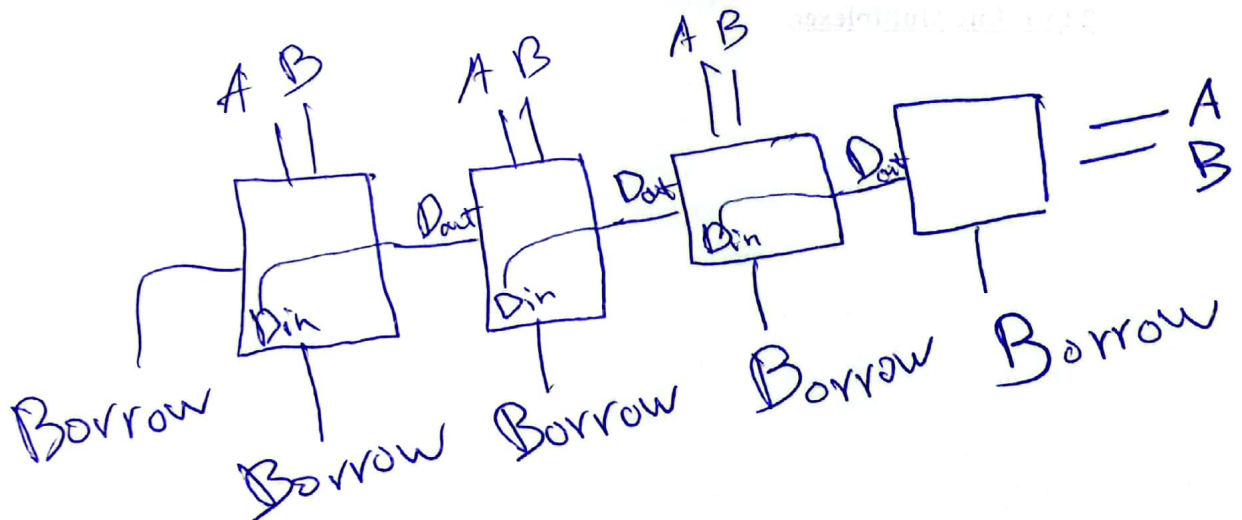
As a result, there are three bits to be handled at the input of a full subtractor, namely the two bits to be subtracted and a borrow bit designated as B_{in} . There are two outputs, namely the DIFFERENCE output D and the BORROW output B_o . The BORROW output bit tells whether the minuend bit needs to borrow a '1' from the next possible higher minuend bit.



Block schematic of full-adder

The truth table for full-subtractor is,

Inputs			Outputs	
A	B	B_{in}	Difference(D)	Borrow(B_{out})
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



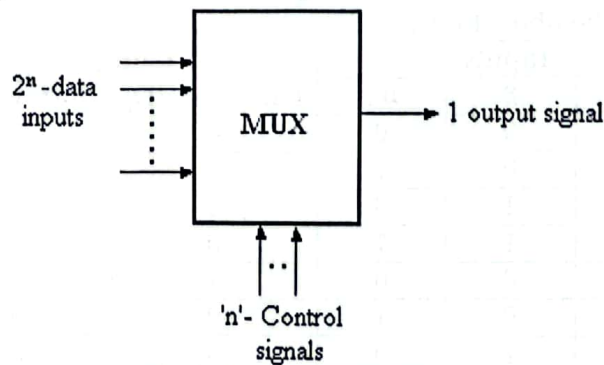
To choose which gate to perform, usually used in smart home.

The circuit has two
When $S=0$, the upper AND
When $S=1$, the lower AND

MULTIPLEXER: (Data Selector)

A **multiplexer** or **MUX**, is a combinational circuit with more than one input line, one output line and more than one selection line. A multiplexer selects binary information present from one of many input lines, depending upon the logic status of the selection inputs, and routes it to the output line. Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected. The multiplexer is often labeled as **MUX** in block diagrams.

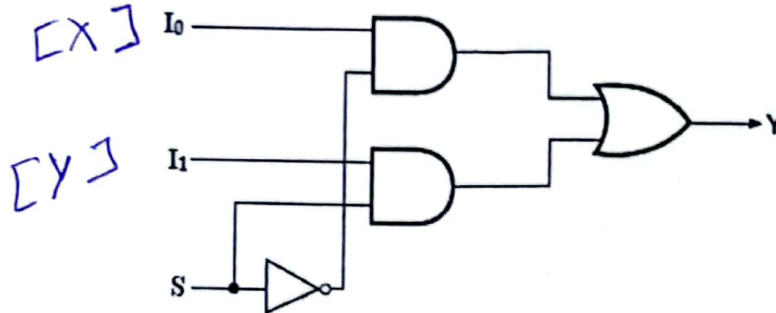
A multiplexer is also called a **data selector**, since it selects one of many inputs and steers the binary information to the output line.



Block diagram of Multiplexer

2-to-1- line Multiplexer:

The circuit has two data input lines, one output line and one selection line, S .
 When $S=0$, the upper AND gate is enabled and I_0 has a path to the output.
 When $S=1$, the lower AND gate is enabled and I_1 has a path to the output.



Logic diagram

The multiplexer acts like an electronic switch that selects one of the two sources.

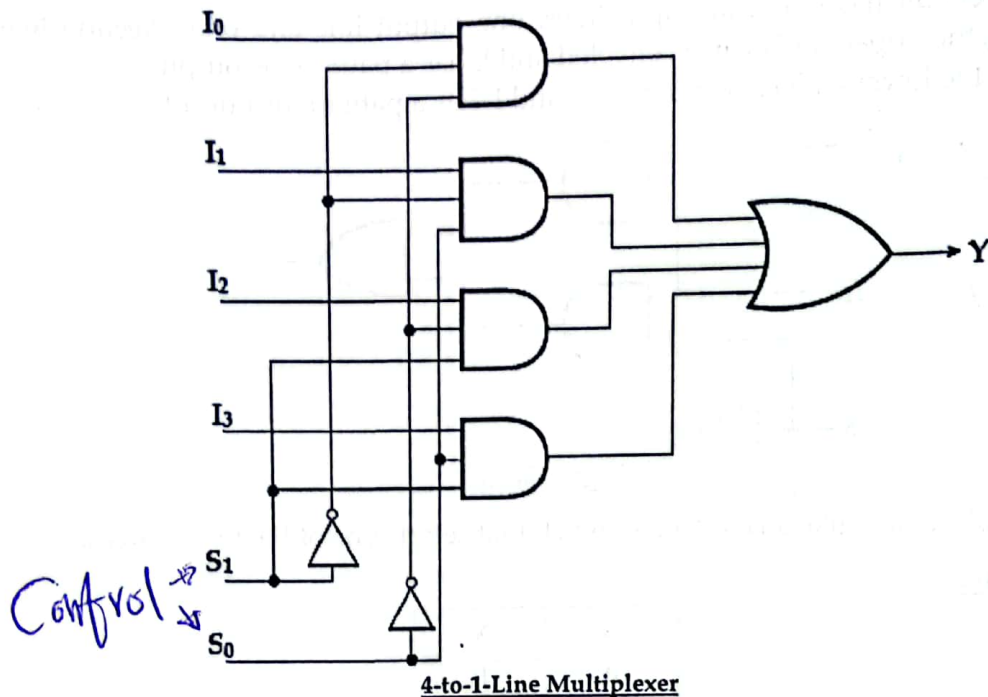
Truth table:

S	Y
0	I_0
1	I_1

4-to-1-line Multiplexer:

A 4-to-1-line multiplexer has four (2^n) input lines, two (n) select lines and one output line. It is the multiplexer consisting of four input channels and information of one of the channels can be selected and transmitted to an output line according to the select inputs combinations. Selection of one of the four input channel is possible by two selection inputs.

Each of the four inputs I_0 through I_3 , is applied to one input of AND gate. Selection lines S_1 and S_0 are decoded to select a particular AND gate. The outputs of the AND gate are applied to a single OR gate that provides the 1-line output.



Function table:

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

To demonstrate the circuit operation, consider the case when $S_1S_0 = 10$. The AND gate associated with input I_2 has two of its inputs equal to 1 and the third input connected to I_2 . The other three AND gates have at least one input equal to 0, which makes their outputs equal to 0. The OR output is now equal to the value of I_2 , providing a path from the selected input to the output.

The data output is equal to I_0 only if $S_1 = 0$ and $S_0 = 0$; $Y = I_0S_1'S_0'$.

The data output is equal to I_1 only if $S_1 = 0$ and $S_0 = 1$; $Y = I_1S_1'S_0$.

The data output is equal to I_2 only if $S_1 = 1$ and $S_0 = 0$; $Y = I_2S_1S_0'$.

The data output is equal to I_3 only if $S_1 = 1$ and $S_0 = 1$; $Y = I_3S_1S_0$.

When these terms are ORed, the total expression for the data output is,

$$Y = I_0S_1'S_0' + I_1S_1'S_0 + I_2S_1S_0' + I_3S_1S_0.$$